

Datenstruktur Queue in Java | LinkedList

Merkmal	Beschreibung
Implementierung	In Java wird die <code>Queue</code> -Schnittstelle oft durch die <code>LinkedList</code> -Klasse implementiert. Die <code>LinkedList</code> verwendet eine doppelt verkettete Liste, um die Queue zu repräsentieren.
Datenstruktur	Eine <code>LinkedList</code> als <code>Queue</code> speichert Elemente in einer Reihenfolge, wobei das erste eingefügte Element auch das erste entfernte ist (FIFO-Prinzip – First In, First Out).
Speicher	Da eine <code>LinkedList</code> eine dynamische Datenstruktur ist, wächst oder schrumpft der Speicherbedarf je nach Anzahl der Elemente, ohne dass eine feste Kapazität erforderlich ist.
Performance	Die <code>LinkedList</code> -Implementierung der <code>Queue</code> bietet konstante Zeitkomplexität ($O(1)$) für das Hinzufügen und Entfernen von Elementen am Kopf und am Ende der Liste.
Thread-Sicherheit	Eine <code>LinkedList</code> als Queue ist nicht thread-sicher. Für parallele Operationen kann eine <code>ConcurrentLinkedQueue</code> oder Synchronisierung verwendet werden.

Methoden

Methode	Beschreibung
<code>offer(E e)</code>	Fügt das Element am Ende der Queue ein. Gibt <code>true</code> zurück, wenn das Element erfolgreich eingefügt wurde, andernfalls <code>false</code> .
<code>add(E e)</code>	Fügt das Element am Ende der Queue ein. Wirft eine <code>IllegalStateException</code> , wenn das Element aufgrund von Kapazitätsbeschränkungen nicht eingefügt werden kann.
<code>poll()</code>	Entfernt und gibt das erste Element der Queue zurück. Gibt <code>null</code> zurück, wenn die Queue leer ist.
<code>remove()</code>	Entfernt und gibt das erste Element der Queue zurück. Wirft eine <code>NoSuchElementException</code> , wenn die Queue leer ist.
<code>peek()</code>	Gibt das erste Element der Queue zurück, ohne es zu entfernen. Gibt <code>null</code> zurück, wenn die Queue leer ist.
<code>element()</code>	Gibt das erste Element der Queue zurück, ohne es zu entfernen. Wirft eine <code>NoSuchElementException</code> , wenn die Queue leer ist.
<code>size()</code>	Gibt die Anzahl der Elemente in der Queue zurück.
<code>isEmpty()</code>	Gibt <code>true</code> zurück, wenn die Queue leer ist, andernfalls <code>false</code> .

Methode	Beschreibung
clear()	Entfernt alle Elemente aus der Queue.
iterator()	Gibt einen Iterator zurück, um die Elemente der Queue in einer Reihenfolge zu durchlaufen.